```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv('/content/iris.csv')
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Next steps: Generate code with df New interactive sheet

```
df.tail()
```

	sepal_length	sepal_width	petal_length	petal_width	species
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
df.dtypes
```

	0
sepal_length	float64
sepal_width	float64
petal_length	float64
petal_width	float64
species	object

dtype: object

```
df.species.unique()
```

```
array(['setosa', 'versicolor', 'virginica'], dtype=object)
```

```
mapi={'setosa':0,'versicolor':1,'virginica':2}
df=df.replace({'species':mapi})
df.head()
```

```
/tmp/ipykernel_9510/3903299706.py:2: FutureWarning: Downcasting behavior in `
  df=df.replace({'species':mapi})
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Next steps: Generate code with df New interactive sheet

```
df.dtypes
```

	0
sepal_length	float64
sepal_width	float64
petal_length	float64
petal_width	float64
species	int64

dtype: object

```
df.isnull().sum()
```

	0
sepal_length	0
sepal_width	0
petal_length	0
petal_width	0
species	0

dtype: int64

```
x,y=df.iloc[:,:-1],df.iloc[:,-1]
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.35,random_sta
```

```
from sklearn.naive_bayes import GaussianNB
model=GaussianNB()
model.fit(x_train,y_train)
```

```
▾ GaussianNB
GaussianNB()
```

```
y_pred=model.predict(x_test)
print(y_pred)
```

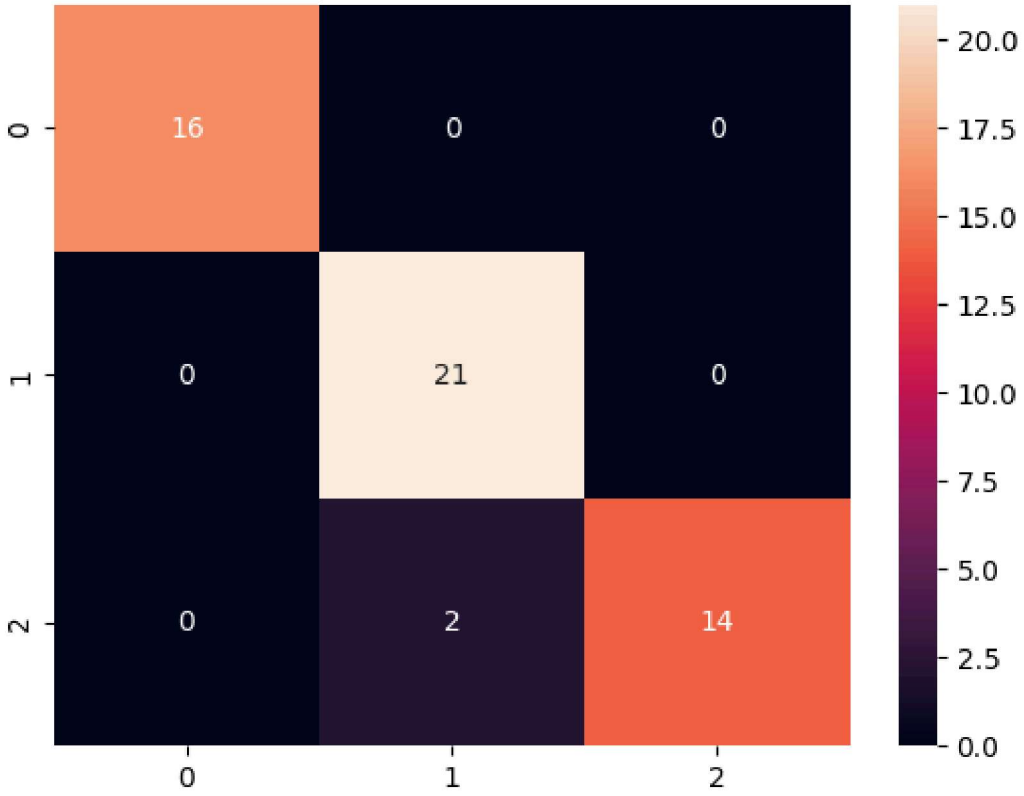
```
[2 1 0 2 0 2 0 1 1 1 1 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
 1 1 1 2 0 2 0 0 1 2 2 1 2 1 2 1]
```

```
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[16  0  0]
 [ 0 21  0]
 [ 0  2 14]]
```

```
import seaborn as sns
sns.heatmap(cm,annot=True)
```

```
<Axes: >
```




```
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1

overall_accuracy = accuracy_score(y_test, y_pred)

overall_error_rate = 1 - overall_accuracy


overall_precision_macro = precision_score(y_test, y_pred, average='macro')
overall_recall_macro = recall_score(y_test, y_pred, average='macro')
overall_f1_macro = f1_score(y_test, y_pred, average='macro')

print("\nOverall Metrics:")
print(f"  Overall Accuracy: {overall_accuracy:.4f}")
print(f"  Overall Error Rate: {overall_error_rate:.4f}")
print(f"  Overall Precision : {overall_precision_macro:.4f}")
print(f"  Overall Recall : {overall_recall_macro:.4f}")
print(f"  Overall F1-Score : {overall_f1_macro:.4f}")
```

```
Overall Metrics:
  Overall Accuracy: 0.9623
  Overall Error Rate: 0.0377
  Overall Precision : 0.9710
```

```
Overall Recall : 0.9583
Overall F1-Score : 0.9626
```